



CẤU TRÚC DỮ LIỆU & GIẢI THUẬT

BÀI 6: Thuật toán đường đi ngắn nhất

PHẦN 1



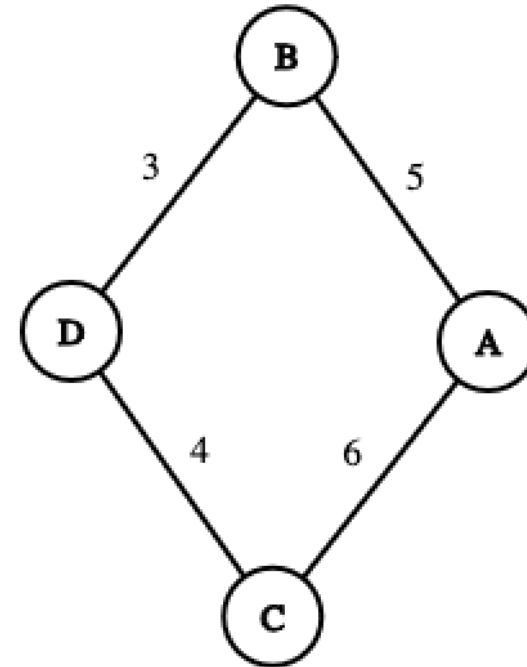
Bài toán Shortest Path

Định nghĩa: Cho đồ thị có trọng số $G = (V, E, W)$, tìm đường đi có tổng trọng số nhỏ nhất từ đỉnh nguồn s đến đỉnh đích t .

Đường đi ngắn nhất từ A đến D?

Các phương án:

- $A \rightarrow B \rightarrow D: 5 + 3 = 8$
- $A \rightarrow C \rightarrow D: 6 + 4 = 10$
- Shortest: $A \rightarrow B \rightarrow D$ (8)



Ứng dụng: GPS navigation, mạng máy tính, game AI, logistics...

Các loại bài toán Shortest Path

- Single-Source Shortest Path (SSSP): Tìm đường đi ngắn nhất từ 1 đỉnh nguồn đến tất cả các đỉnh khác
- Single-Pair Shortest Path: Tìm đường đi ngắn nhất giữa 2 đỉnh cụ thể ($s \rightarrow t$)
- All-Pairs Shortest Path: Tìm đường đi ngắn nhất giữa mọi cặp đỉnh
- Buổi hôm nay: Tập trung vào SSSP với Dijkstra's Algorithm

Dijkstra's Algorithm - Tổng quan

Phát minh: Edsger W. Dijkstra (1956, Hà Lan)

Đặc điểm:

- Tìm đường đi ngắn nhất từ 1 đỉnh nguồn đến tất cả đỉnh khác
- Yêu cầu: Trọng số các cạnh không âm (≥ 0)
- Dựa trên nguyên lý Greedy: chọn đỉnh gần nhất chưa xử lý

Ý tưởng chính: Duy trì bảng khoảng cách $d[v]$ và liên tục cập nhật với đường đi tốt hơn

Các khái niệm cốt lõi

1. Bảng khoảng cách $d[v]$:

- $d[v]$ = khoảng cách ngắn nhất từ nguồn s đến đỉnh v
- Ban đầu: $d[s] = 0$, $d[v] = \infty$ với $v \neq s$
- Liên tục được cập nhật trong quá trình chạy

2. Relaxation (Nới lỏng cạnh): Với cạnh $u \rightarrow v$ có trọng số w :

```
Nếu  $d[u] + w < d[v]$ :  
     $d[v] = d[u] + w$  # Tìm được đường đi tốt hơn!  
     $parent[v] = u$    # Lưu đỉnh trước  $v$ 
```

3. Priority Queue:

Lưu các đỉnh chưa xử lý, sắp xếp theo $d[v]$ tăng dần

→ Luôn chọn đỉnh có khoảng cách nhỏ nhất

4. Visited Set:

Tập các đỉnh đã xử lý xong (khoảng cách đã tối ưu)

Giải thích Relaxation (Nới lỏng)

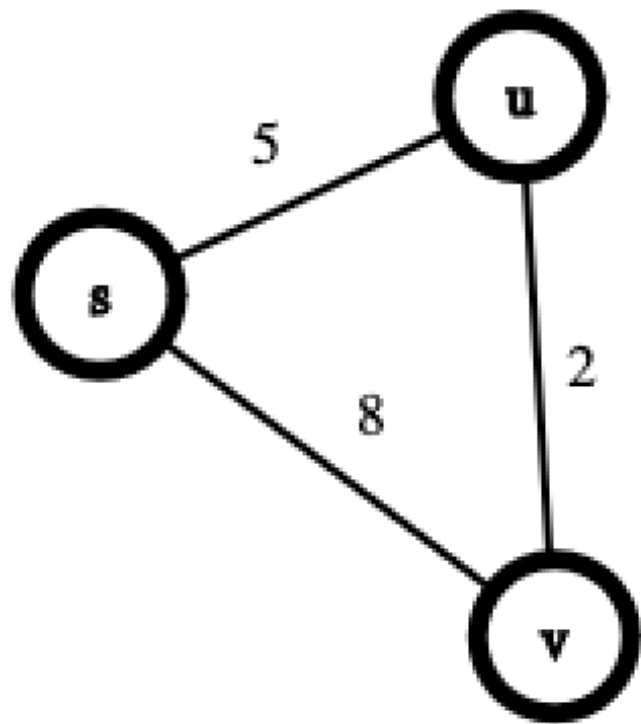
Tại sao gọi là “relaxation”? Giảm (nới lỏng) ước lượng khoảng cách từ nguồn đến đỉnh v

Bước 1: Ban đầu

$$d[s] = 0$$

$$d[u] = 5$$

$$d[v] = 8$$



Bước 2: Xét cạnh $u \rightarrow v$ (trọng số 2)

- Đường đi mới: $s \rightarrow u \rightarrow v = d[u] + 2 = 5 + 2 = 7$
- So sánh: $7 < 8$ (đường cũ)
- Relaxation: $d[v] = 7$ (cập nhật!)

Kết quả: Tìm được đường đi tốt hơn từ s đến v !

Thuật toán Dijkstra

- $d[v]$: Khoảng cách ngắn nhất từ source đến v
- $parent[v]$: Đỉnh trước v trên đường đi ngắn nhất

```
Dijkstra(graph, source):
```

```
1. Khởi tạo:
```

- $d[source] = 0$
- $d[v] = \infty$ với mọi $v \neq source$
- Thêm tất cả đỉnh vào priority queue (theo $d[v]$)
- $visited = \{\}$

```
2. While priority queue không rỗng:
```

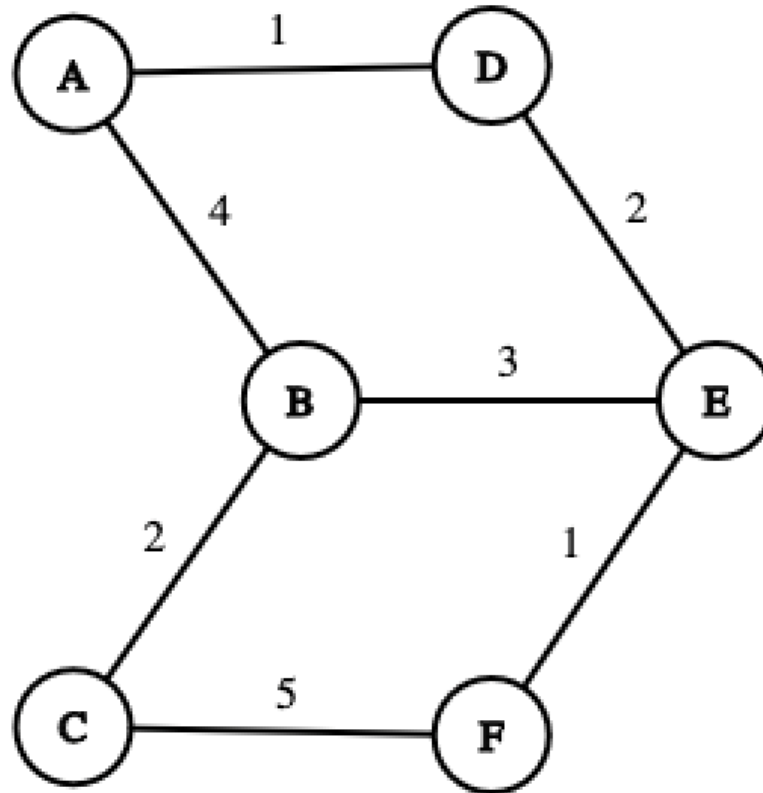
- Lấy đỉnh u có $d[u]$ nhỏ nhất (từ pq)
- Nếu u đã $visited \rightarrow skip$
- Đánh dấu u là $visited$

```
d. Với mỗi neighbor  $v$  của  $u$ :
```

- Tính $distance_new = d[u] + weight(u, v)$
- Nếu $distance_new < d[v]$:
 - $\rightarrow d[v] = distance_new$ (Relaxation!)
 - $\rightarrow parent[v] = u$
 - $\rightarrow$ Thêm $(d[v], v)$ vào priority queue

```
3. Return  $d[], parent[]$ 
```

Tìm đường đi ngắn nhất từ A đến tất cả các đỉnh



Bảng theo dõi $d[v]$ qua từng bước:

Bước	Chọn đỉnh	$d[A]$	$d[B]$	$d[C]$	$d[D]$	$d[E]$	$d[F]$	Visited
0	-	0	∞	∞	∞	∞	∞	{ }
1	A	0	4	∞	1	∞	∞	{ A }
2	D	0	4	∞	1	3	∞	{ A, D }
3	E	0	4	∞	1	3	4	{ A, D, E }
4	B	0	4	6	1	3	4	{ A, D, E, B }
5	F	0	4	6	1	3	4	{ A, D, E, B, F }
6	C	0	4	6	1	3	4	{ A, D, E, B, F, C } ¹²

Kết quả cuối cùng

- $A \rightarrow B$: $[A, B]$ với chi phí 4
- $A \rightarrow C$: $[A, B, C]$ với chi phí 6
- $A \rightarrow D$: $[A, D]$ với chi phí 1
- $A \rightarrow E$: $[A, D, E]$ với chi phí 3
- $A \rightarrow F$: $[A, D, E, F]$ với chi phí 4

Cài đặt Dijkstra với Python

```
def reconstruct_path(parents, source, target):
    """
    Reconstruct đường đi từ parent table
    """
    path = []
    current = target

    while current is not None:
        path.append(current)
        current = parents[current]

    path.reverse()

    # Kiểm tra có đường đi không
    if path[0] != source:
        return None # Không có đường đi

    return path
```

```
import heapq

def dijkstra(graph, source):
    """
    Dijkstra's Algorithm - Tìm đường đi ngắn nhất
    """
    # Khởi tạo
    distances = {vertex: float('inf') for vertex in graph}
    distances[source] = 0
    parents = {vertex: None for vertex in graph}
    pq = [(0, source)]
    visited = set()

    # Vòng lặp chính
    while pq:
        current_distance, current_vertex = heapq.heappop(pq)

        if current_vertex in visited:
            continue
        visited.add(current_vertex)

        # Relaxation
        for neighbor, weight in graph[current_vertex]:
            distance_new = current_distance + weight

            if distance_new < distances[neighbor]:
                distances[neighbor] = distance_new
                parents[neighbor] = current_vertex
                heapq.heappush(pq, (distance_new, neighbor))

    return distances, parents
```

Tại sao Dijkstra đúng?

Định lý: Với đồ thị có trọng số không âm, Dijkstra luôn tìm được đường đi ngắn nhất.

Tại mọi thời điểm, với mọi đỉnh u đã visited: $d[u]$ là khoảng cách ngắn nhất từ source đến u .

Điều gì xảy ra với trọng số âm?

Dijkstra KHÔNG hoạt động với trọng số âm!

Nguyên nhân:

- Greedy choice sai khi có trọng số âm
- Đường đi qua đỉnh “xa” có thể ngắn hơn nhờ cạnh âm

Giải pháp: Dùng Bellman-Ford ($O(VE)$) thay vì Dijkstra

Ứng dụng thực tế của Dijkstra

1. GPS Navigation (Bản đồ):

- Tìm đường đi ngắn nhất từ A đến B
- Trọng số = khoảng cách hoặc thời gian
- Google Maps, Waze, Apple Maps

2. Network Routing:

- Tìm đường truyền dữ liệu tốt nhất
- OSPF (Open Shortest Path First) protocol
- Trọng số = latency, bandwidth

3. Robotics:

- Path planning cho robot
- Tránh vật cản, tối ưu hóa năng lượng

4. Game AI:

- Tìm đường cho NPC (Non-Player Character)
- A* algorithm (biến thể của Dijkstra)

5. Airline Routes:

- Tìm chuyến bay rẻ nhất, nhanh nhất

Các biến thể của Dijkstra

1. Single-target Dijkstra:

- Dừng ngay khi đến đích (thay vì duyệt hết)
- Tối ưu hóa khi chỉ cần tìm 1 đường đi

2. Bidirectional Dijkstra:

- Chạy 2 Dijkstra đồng thời từ source và target
- Gặp nhau ở giữa $\rightarrow$ Nhanh hơn ~ 2 lần

3. A Algorithm:*

- Dijkstra + Heuristic function
- Ưu tiên các đỉnh “gần đích” hơn
- Nhanh hơn Dijkstra trong nhiều trường hợp

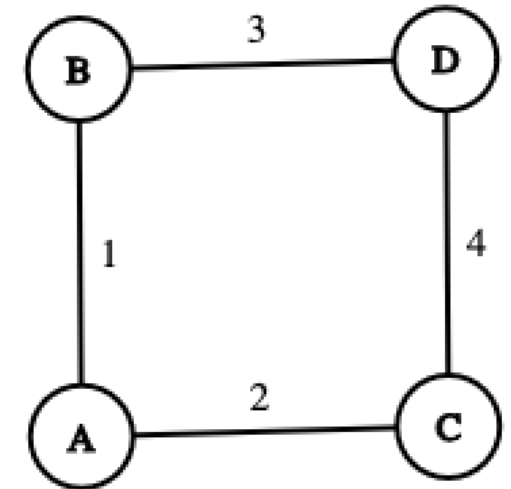
So sánh Dijkstra vs BFS

Tiêu chí	BFS	Dijkstra
Trọng số	Không có (hoặc = 1)	Có trọng số không âm
Cấu trúc	Queue (FIFO)	Priority Queue (Min-Heap)
Độ phức tạp	$O(V + E)$	$O((V+E) \log V)$
Đường đi	Ít đỉnh nhất	Chi phí nhỏ nhất
Khi nào dùng	Unweighted graph	Weighted graph

- BFS từ A: $A \rightarrow B \rightarrow C \rightarrow D$ (3 cạnh)
- Dijkstra từ A: $A \rightarrow B \rightarrow D$ (chi phí $1+3=4$) hoặc $A \rightarrow C \rightarrow D$ (chi phí $2+4=6$)

Kết luận:

- BFS tìm đường ít đỉnh nhất
- Dijkstra tìm đường chi phí nhỏ nhất
- Với trọng số = 1 $\rightarrow$ BFS và Dijkstra cho cùng kết quả



PHẦN 2



Giả sử có 6 chi nhánh trung tâm, cần kéo dây mạng nối tất cả lại với nhau:
Mỗi cặp chi nhánh có chi phí kéo dây khác nhau (km, tiền, thời gian).

Mục tiêu:

- Tất cả chi nhánh đều được nối (từ bất kỳ điểm nào cũng đi được đến điểm khác).
- Tổng chi phí nhỏ nhất.
- Không cần vòng tròn dư thừa (không cần “đường vòng” tốn tiền).

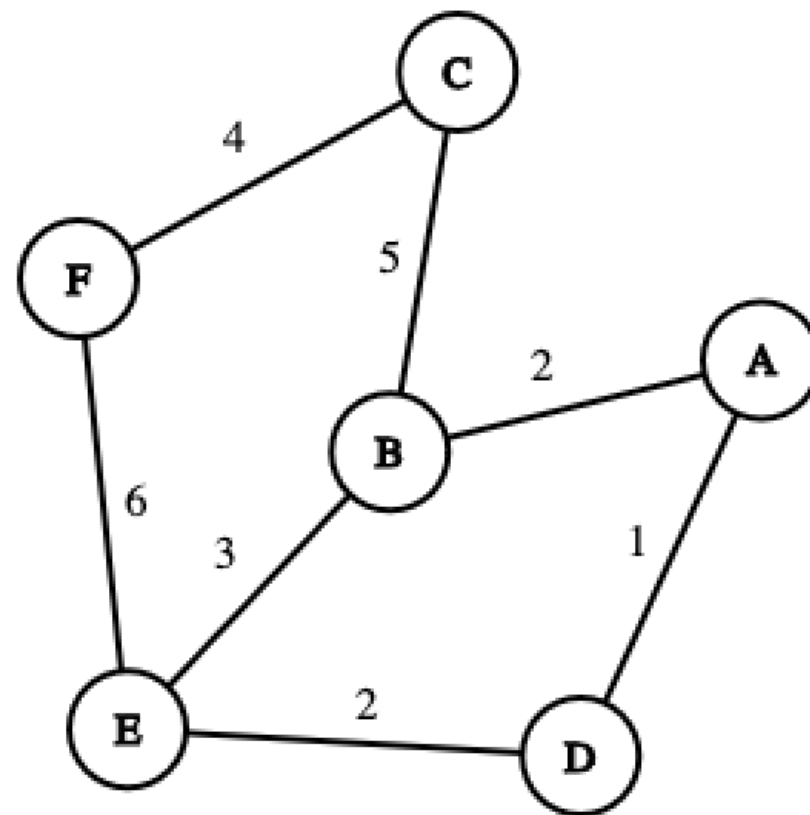
Đây chính là bài toán cây khung nhỏ nhất – MST.

Đỉnh: A, B, C, D, E, F là chi nhánh.

Số trên đường là chi phí kéo dây.

Ta muốn chọn một số đường sao cho:

- Nối tất cả A, B, C, D, E, F.
- Không tạo vòng.
- Tổng chi phí nhỏ nhất.



Ý tưởng trực quan Kruskal

- Viết ra tất cả các đường (cạnh) với chi phí của nó.
- Sắp xếp các đường này từ rẻ nhất $\rightarrow$ đắt nhất.
- Bắt đầu từ đường rẻ nhất:
 - Nếu nối 2 điểm chưa được nối với nhau $\rightarrow$ lấy.
 - Nếu tạo vòng lặp (đi vòng vòng về chỗ cũ) $\rightarrow$ bỏ qua.
- Làm cho đến khi tất cả các chi nhánh có thể đi tới nhau.

Kết quả: một “bộ xương” mạng lưới gọi là MST.

Bài toán: nhận diện “tạo vòng”

Làm sao biết “thêm đường này sẽ tạo vòng” mà không phải vẽ lại cả đồ thị và mắt nhìn đoán?

Ví dụ:

- Đã chọn $A-B$, $B-C$.
- Bây giờ xem xét cạnh $A-C$.
- Thêm $A-C$ vào ta có $A-B-C-A \rightarrow$ vòng.

Ta cần một cách:

- A và C bây giờ có cùng nằm trong một “nhóm đã nối sẵn” chưa?
- Nếu đã nằm chung nhóm $\rightarrow$ thêm cạnh nữa sẽ tạo vòng.
- Nếu đang ở 2 nhóm khác nhau $\rightarrow$ nối chúng là hợp lý.

Cấu trúc dữ liệu làm việc này chính là Union–Find (DSU).

Union–Find

Khi xem xét cạnh mới (C, E) :

- Nếu C thuộc nhóm 1, E thuộc nhóm 2 $\rightarrow$ khác nhóm $\rightarrow$ được nối $\rightarrow$ gộp nhóm 1 và 2.
- Nếu cạnh (A, C) mà A và C đều trong nhóm 1 $\rightarrow$ cùng nhóm $\rightarrow$ bỏ, vì nối vào là tạo vòng.

Union–Find chỉ làm đúng 2 việc:

- Hỏi: “Bạn này đang thuộc nhóm nào?” $\rightarrow$ find(u)
- Nối hai nhóm lại với nhau $\rightarrow$ union(u, v)

Union-Find: trực quan bằng cây

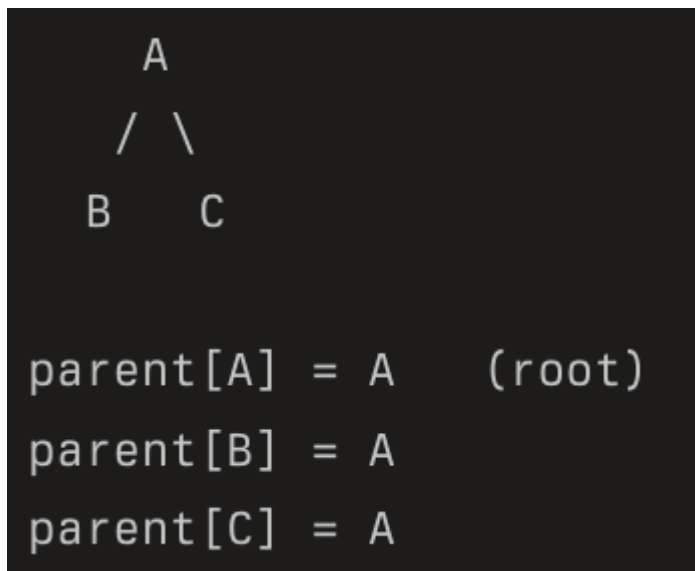
Ta lưu nhóm bằng cách cho mỗi nhóm là một cây nhỏ:

- Mỗi node có một parent.
- Root của cây là “trưởng nhóm”, tự trở về mình.

Ví dụ: nhóm {A, B, C}:

□ $\text{find}(B) \rightarrow A$

□ $\text{find}(C) \rightarrow A$



Code DSU

```
def make_set(vertices):  
    parent = {}  
    for v in vertices:  
        parent[v] = v # mỗi đỉnh là root của chính nó  
    return parent  
  
def find(parent, v):  
    # Leo lên đến khi gặp root  
    while parent[v] != v:  
        v = parent[v]  
    return v  
  
def union(parent, a, b):  
    root_a = find(parent, a)  
    root_b = find(parent, b)  
    if root_a != root_b:  
        parent[root_b] = root_a # cho root_b làm con của root_a
```

Kruskal: bước chuẩn bị

sort cạnh theo trọng số:

```
edges_sorted = sorted(edges, key=lambda e: e[0])  
# → [(1, A, B), (2, B, D), (2, D, E), (3, B, C), (4, A, C), (5, C, E)]
```

make_set cho tất cả đỉnh:

```
parent = make_set(vertices)  
mst = []  
total = 0
```

```
vertices = ['A', 'B', 'C', 'D', 'E']  
edges = [  
    (1, 'A', 'B'),  
    (4, 'A', 'C'),  
    (3, 'B', 'C'),  
    (2, 'B', 'D'),  
    (5, 'C', 'E'),  
    (2, 'D', 'E'),  
]  
# Mỗi cạnh: (weight, u, v)
```


Duyệt edges_sorted

Bước	Cạnh (w,u,v)	find(u)	find(v)	Quyết định	MST hiện tại
1	(1,A,B)	A	B	khác nhóm $\rightarrow$ lấy	(A,B,1)
2	(2,B,D)	A	D	$A \neq D \rightarrow$ lấy	(A,B,1), (B,D,2)
3	(2,D,E)	A	E	$A \neq E \rightarrow$ lấy	+ (D,E,2)
4	(3,B,C)	A	C	$A \neq C \rightarrow$ lấy	+ (B,C,3)
5	(4,A,C)	A	A	cùng nhóm $\rightarrow$ bỏ	giữ nguyên
6	(5,C,E)	A	A	cùng nhóm $\rightarrow$ bỏ	giữ nguyên

Khi có đủ 4 cạnh (vì 5 đỉnh $\rightarrow$ MST có 4 cạnh) thì dừng.

MST: (A,B,1), (B,D,2), (D,E,2), (B,C,3), tổng = 8.

Code Kruskal

find trả về “trưởng nhóm”.

Khác trưởng nhóm $\rightarrow$ lấy cạnh,
rồi union để gộp nhóm.

```
def kruskal_mst(vertices, edges):  
    # Sort cạnh theo trọng số  
    edges_sorted = sorted(edges, key=lambda e: e[0])  
  
    # Khởi tạo DSU  
    parent = make_set(vertices)  
  
    mst = []  
    total_weight = 0  
  
    # Duyệt từng cạnh  
    for w, u, v in edges_sorted:  
        root_u = find(parent, u)  
        root_v = find(parent, v)  
  
        # Nếu khác nhóm  $\rightarrow$  không tạo chu trình  
        if root_u != root_v:  
            mst.append((u, v, w))  
            total_weight += w  
            union(parent, root_u, root_v)  
  
        # Nếu đủ  $|V|-1$  cạnh  $\rightarrow$  dừng  
        if len(mst) == len(vertices) - 1:  
            break  
  
    return mst, total_weight
```

So sánh cảm giác Dijkstra vs Kruskal

Dijkstra:

- Giải đường đi ngắn nhất từ 1 đỉnh tới các đỉnh khác.
- Dùng priority queue (heap).

Kruskal:

- Giải cây khung nhỏ nhất cho toàn bộ đồ thị.
- Không tìm đường đi giữa 2 đỉnh cụ thể, mà tìm mạng “xương sống” rẻ nhất.
- Dùng DSU (Union–Find).



Kết thúc